



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**  
**(P-132)**

**LOVELY PROFESSIONAL UNIVERSITY**

**COURSE TITLE: - CLUSTER COMPUTING**

**COURSE CODE: - INT-315**

**TITLE: - Sales Trend Prediction**

|                        |              |
|------------------------|--------------|
| <b>Name</b>            | Andol Saketh |
| <b>Registration No</b> | 12221210     |
| <b>Roll No</b>         | 32           |
| <b>Section</b>         | K22YE        |

**Submitted By: -**

**Submitted To: -**

Andol Saketh

Dr.Richa

12221210

## INTRODUCTION: -

The goal of this project is to analyse and predict sales trends using real-world business data.

Sales forecasting is essential for effective decision-making, inventory control, and performance analysis.

By studying sales patterns over time, this project aims to uncover trends and generate predictions for future sales using machine learning techniques.

The model applies time-series analysis with **lag features** (previous months' sales) and uses **Linear Regression** in PySpark to forecast future monthly sales.

Predictive insights from this analysis can help organizations in demand planning, financial forecasting, and strategic growth management.

## 2. Dataset Details

The dataset used in this project is the Global Superstore Sales Dataset (4 years) — a popular open-source dataset from Kaggle. It contains detailed transactional sales data across multiple categories, regions, and customers.

Each record represents an individual order, including sales amount, date, and related metadata.

Dataset File: train.csv

Location: C:/Users/saket/Documents/SAKETH/archive/train.csv

Dataset Source: Kaggle – Global Superstore 4-Year Dataset

## SOURCE CODE: -

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, lag, sum as _sum, to_date, date_format, regexp_replace,
    when, length, expr, trim, isnan
)
from pyspark.sql.window import Window
```

```

from pyspark.ml.feature import VectorAssembler

from pyspark.ml.regression import LinearRegression

from pyspark.ml.evaluation import RegressionEvaluator

import os

```

```

spark = SparkSession.builder.appName("SalesTrendPrediction_INT315_Final").getOrCreate()

print("\nSpark Session Started Successfully!\n")

```

```

base_path = "C:/Users/saket/Documents/SAKETH/archive"

data_path = os.path.join(base_path, "train.csv")

output_folder = os.path.join(base_path, "Predicted_Sales_Output")

```

```

print(f"Current Working Directory: {os.getcwd()}")

print(f"Output will be saved to: {output_folder}\n")

```

```

df = spark.read.csv(data_path, header=True, inferSchema=True)

```

```

print("Dataset Loaded Successfully!")

print("Columns:", df.columns)

df.show(5, truncate=False)

```

```

product Name', 'Sales']
> df.show(5, truncate=False)

```

| Order ID       | Order ID   | Order Date | Ship Date      | Ship Mode | Customer ID     | Customer Name | Segment       | Country         | City       | State | Postal Code | Region          | Product ID      | Category | Sub-Category |
|----------------|------------|------------|----------------|-----------|-----------------|---------------|---------------|-----------------|------------|-------|-------------|-----------------|-----------------|----------|--------------|
| CA-2017-152156 | 08/11/2017 | 11/11/2017 | Second Class   | CG-12520  | Claire Gute     | Consumer      | United States | Henderson       | Kentucky   | 42420 | South       | FUR-BO-10001798 | Furniture       | Bookcase |              |
| CA-2017-152156 | 08/11/2017 | 11/11/2017 | Second Class   | CG-12520  | Claire Gute     | Consumer      | United States | Henderson       | Kentucky   | 42420 | South       | FUR-CH-10000454 | Furniture       | Chairs   |              |
| CA-2017-138688 | 12/06/2017 | 16/06/2017 | Second Class   | DV-13045  | Darrin Van Huff | Corporate     | United States | Los Angeles     | California | 90036 | West        | OFF-LA-10000240 | Office Supplies | Labels   |              |
| US-2016-108966 | 11/10/2016 | 18/10/2016 | Standard Class | SO-20335  | Sean O'Donnell  | Consumer      | United States | Fort Lauderdale | Florida    | 33311 | South       | FUR-TA-10000577 | Furniture       | Tables   |              |
| US-2016-108966 | 11/10/2016 | 18/10/2016 | Standard Class | SO-20335  | Sean O'Donnell  | Consumer      | United States | Fort Lauderdale | Florida    | 33311 | South       | OFF-ST-10000760 | Office Supplies | Storage  |              |

```

df = df.withColumnRenamed("Order Date", "OrderDate")

df = df.withColumn("OrderDate", to_date(col("OrderDate"), "d/M/yyyy"))

df = df.filter(col("OrderDate").isNotNull())

```

```

print("\nParsed OrderDate Sample:")

```

```
df.select("OrderDate").show(5, truncate=False)
```

```
>> print( "Parsed OrderDate Sample: ")
Parsed OrderDate Sample:
>> df.select("OrderDate").show(5, truncate=False)
-----+
OrderDate |
-----+
2017-11-08 |
2017-11-08 |
2017-06-12 |
2016-10-11 |
2016-10-11 |
-----+
only showing top 5 rows
>>
```

```
df = df.withColumn("Sales_str", col("Sales").cast("string"))
```

```
df = df.withColumn("Sales_str", regexp_replace(col("Sales_str"), r'[,\s\u00A0]+', ''))
```

```
df = df.withColumn(
    "Sales_cleaned",
    when(
        (col("Sales_str").isNull()) |
        (length(col("Sales_str")) == 0) |
        (col("Sales_str").rlike("(?i)^null$")) |
        (col("Sales_str").rlike("(?i)^nan$")) |
        (col("Sales_str").rlike("(?i)^n/?a$")),
        None
    ).otherwise(col("Sales_str"))
)
```

```
df = df.withColumn("Sales_extracted", regexp_replace(col("Sales_cleaned"), "[^0-9.]", ''))
```

```
df = df.withColumn("SalesNumeric", expr("try_cast(Sales_extracted AS double)"))
```

```
df = df.filter(col("SalesNumeric").isNotNull())
```

```
df = df.select("OrderDate", col("SalesNumeric").alias("Sales")).orderBy("OrderDate")
```

```
print("\nCleaned Sales Data:")
```

```
df.show(5, truncate=False)
```

```
>>> print("\nCleaned Sales Data:")
```

Cleaned Sales Data:

```
>>> df.show(5, truncate=False)
```

```
+-----+-----+
|OrderDate|Sales|
+-----+-----+
|2015-01-03|200.0|
|2015-01-04|3.54|
|2015-01-04|11.784|
|2015-01-04|272.736|
|2015-01-05|19.536|
+-----+-----+
only showing top 5 rows
```

```
monthly_sales = (
    df.groupBy(date_format(col("OrderDate"), "yyyy-MM").alias("YearMonth"))
        .agg(_sum("Sales").alias("Total_Sales"))
        .orderBy("YearMonth")
)
```

```
monthly_sales = monthly_sales.withColumn("Total_Sales", expr("try_cast(Total_Sales AS double)"))
```

```
monthly_sales = monthly_sales.filter(col("Total_Sales").isNotNull())
```

```
print("\nMonthly Aggregated Sales:")
```

```
monthly_sales.show(8, truncate=False)
```

```
Monthly Aggregated Sales:
>>> monthly_sales.show(8, truncate=False)
+-----+-----+
|YearMonth|Total_Sales|
+-----+-----+
|2015-01|14430.160999999996|
|2015-02|210633.816|
|2015-03|55040.986999999994|
|2015-04|27751.070999999985|
|2015-05|23630.682999999994|
|2015-06|34298.347600000016|
|2015-07|33446.022999999994|
|2015-08|27611.58049999998|
+-----+-----+
only showing top 8 rows
```

```
windowSpec = Window.orderBy("YearMonth")
```

```
train_data, test_data = final_df.randomSplit([0.8, 0.2], seed=42)
```

```
print(f"\nData Split Completed: Training={train_data.count()}, Testing={test_data.count()}")
```

```
lr = LinearRegression(featuresCol="features", labelCol="Total_Sales")
```

```
lr_model = lr.fit(train_data)
```

```
print("\nLinear Regression Model Trained Successfully!")
```

```
print("Coefficients:", lr_model.coefficients)
```

```
print("Intercept:", lr_model.intercept)
```

```
>>>
>>> print("\nLinear Regression Model Trained Successfully!")

Linear Regression Model Trained Successfully!
>>> print("Coefficients:", lr_model.coefficients)
Coefficients: [0.16553636997256221,0.11590134864932433]
>>> print("Intercept:", lr_model.intercept)
Intercept: 42683.1214351827
>>>
>>> predictions = lr_model.transform(test_data)
>>>
>>> print("\nPredictions on Test Data:")

Predictions on Test Data:
>>> predictions.select("YearMonth", "Total_Sales", "prediction").orderBy("YearMonth").show(10, truncate=False)
25/11/13 17:20:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition
```

```
predictions = lr_model.transform(test_data)
```

```
print("\nPredictions on Test Data:")
```

```
predictions.select("YearMonth", "Total_Sales", "prediction").orderBy("YearMonth").show(10,
truncate=False)
```

```
25/11/13 17:20:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition
+-----+-----+-----+
|YearMonth|Total_Sales      |prediction      |
+-----+-----+-----+
|2015-06   |34298.347600000016|52974.18354326497|
|2015-10   |34127.941         |60160.19145392915|
|2015-12   |70626.274500000001|65940.49681334713|
|2016-05   |230401.41449999987|49484.783946864205|
|2016-11   |77312.869500000004|52365.194416963786|
|2017-03   |62215.733         |55717.045088418236|
|2017-09   |272206.44690000001|76225.63879458996|
|2018-03   |58730.28379999998|57176.63138942509|
+-----+-----+-----+

>>>
>>> evaluator = RegressionEvaluator(
...     labelCol="Total_Sales", predictionCol="prediction", metricName="rmse"
... )
>>> rmse = evaluator.evaluate(predictions)
25/11/13 17:20:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition
```

```
evaluator = RegressionEvaluator(
```

```
    labelCol="Total_Sales", predictionCol="prediction", metricName="rmse"
```

```

)

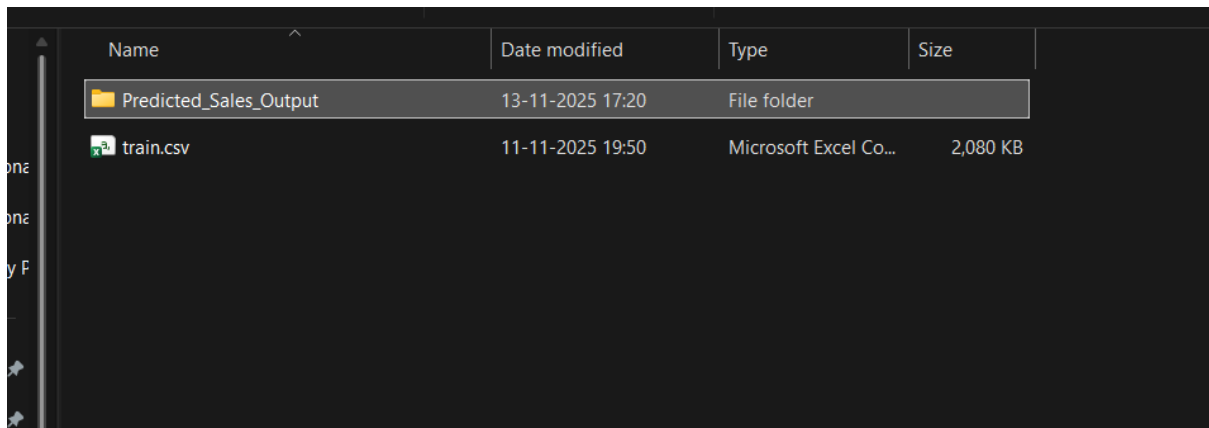
rmse = evaluator.evaluate(predictions)

print(f"\nModel Evaluation Completed!\nRoot Mean Squared Error (RMSE): {rmse:.2f}\n")

predictions.select(
    "YearMonth",
    col("Total_Sales").alias("Actual_Sales"),
    col("prediction").alias("Predicted_Sales")
).coalesce(1).write.mode("overwrite").option("header", True).csv(output_folder)

print(f"Predictions Saved Successfully!")

```



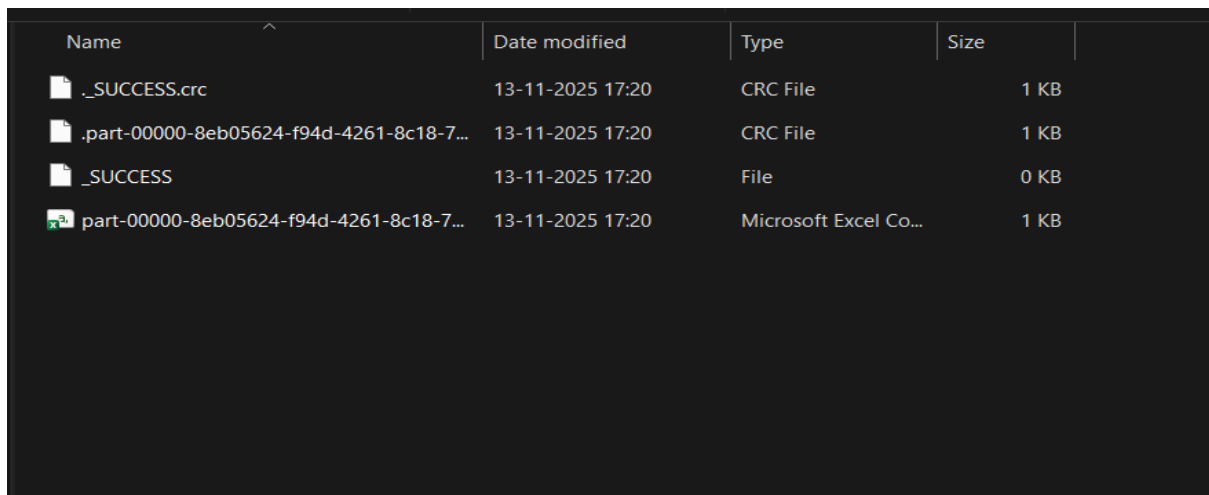
| Name                   | Date modified    | Type                  | Size     |
|------------------------|------------------|-----------------------|----------|
| Predicted_Sales_Output | 13-11-2025 17:20 | File folder           |          |
| train.csv              | 11-11-2025 19:50 | Microsoft Excel Co... | 2,080 KB |

```

print(f"Location: {output_folder}")

print(f"Look for the CSV file inside the folder\n")

```



| Name                                     | Date modified    | Type                  | Size |
|------------------------------------------|------------------|-----------------------|------|
| ._SUCCESS.crc                            | 13-11-2025 17:20 | CRC File              | 1 KB |
| .part-00000-8eb05624-f94d-4261-8c18-7... | 13-11-2025 17:20 | CRC File              | 1 KB |
| ._SUCCESS                                | 13-11-2025 17:20 | File                  | 0 KB |
| part-00000-8eb05624-f94d-4261-8c18-7...  | 13-11-2025 17:20 | Microsoft Excel Co... | 1 KB |

```

spark.stop()

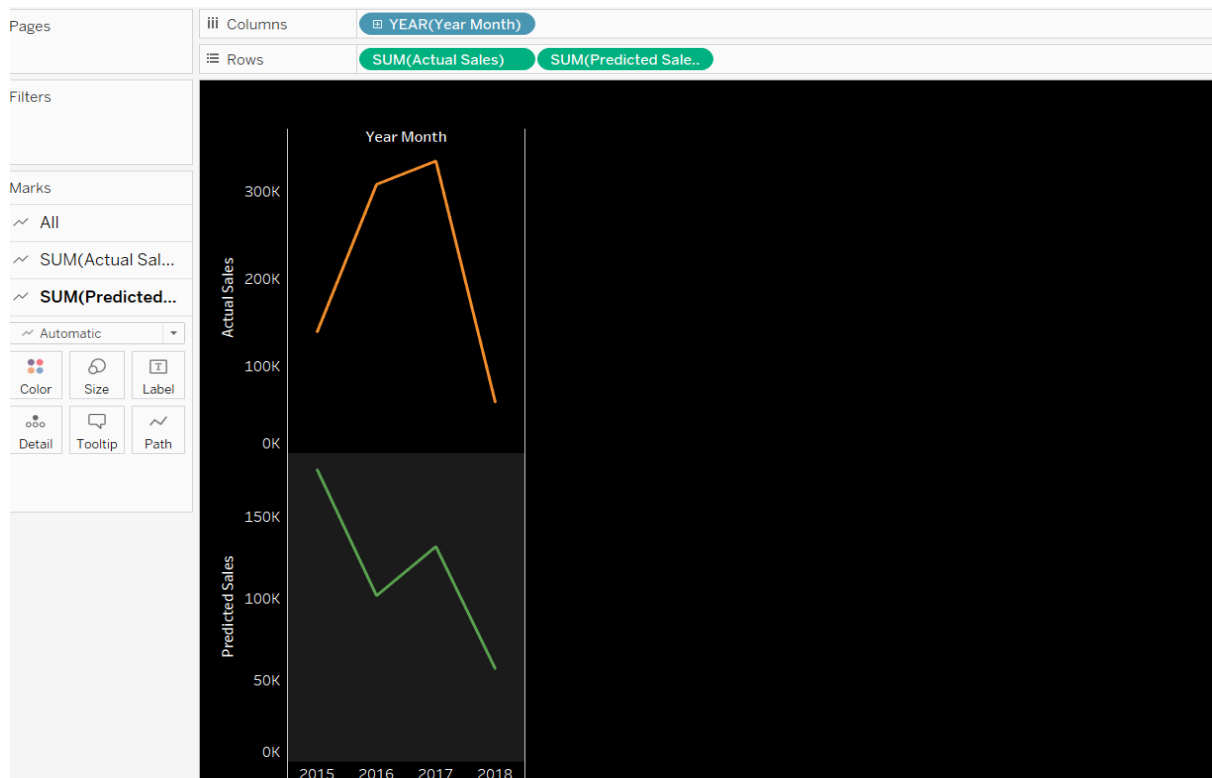
print("Spark Session Stopped. Project Completed Successfully!\n")

```



## DASHBOARD :-

### 1. Actual vs Predicted Sales Line Chart

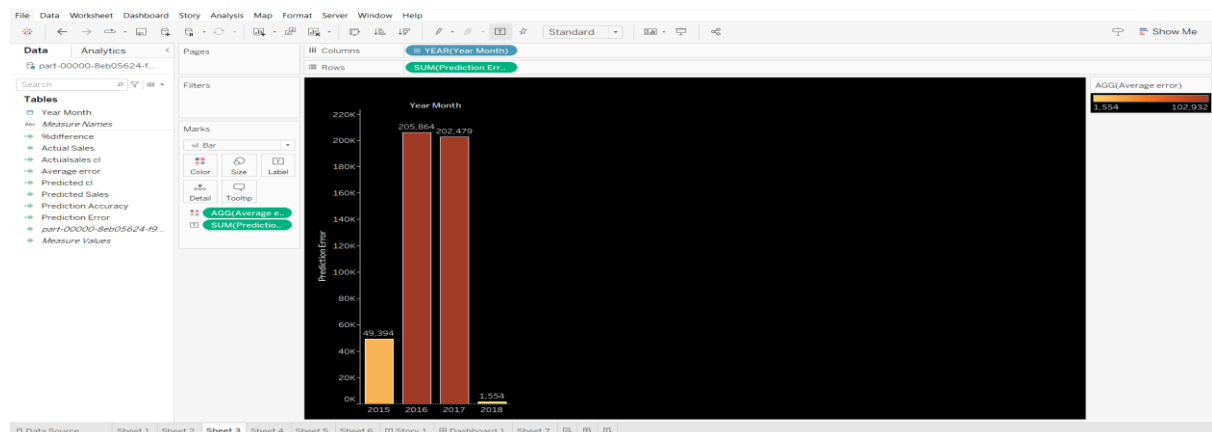


### 2. Prediction Error

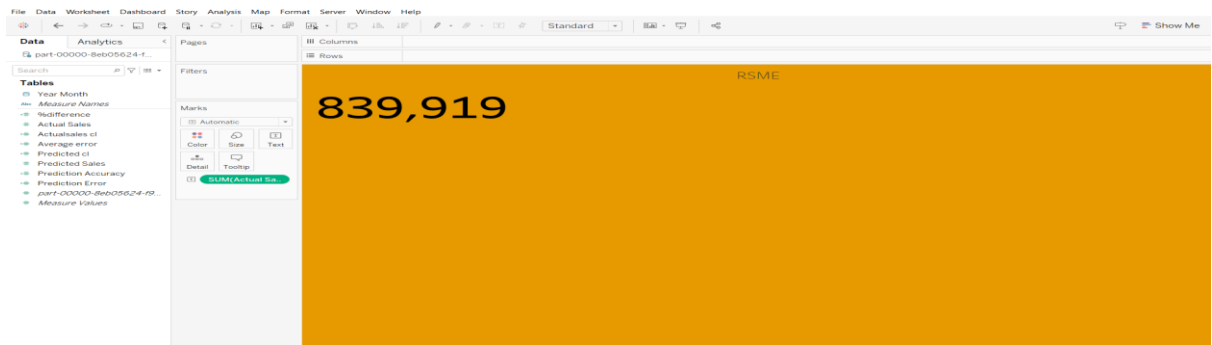
Create a calculated field named prediction error

And write the formula

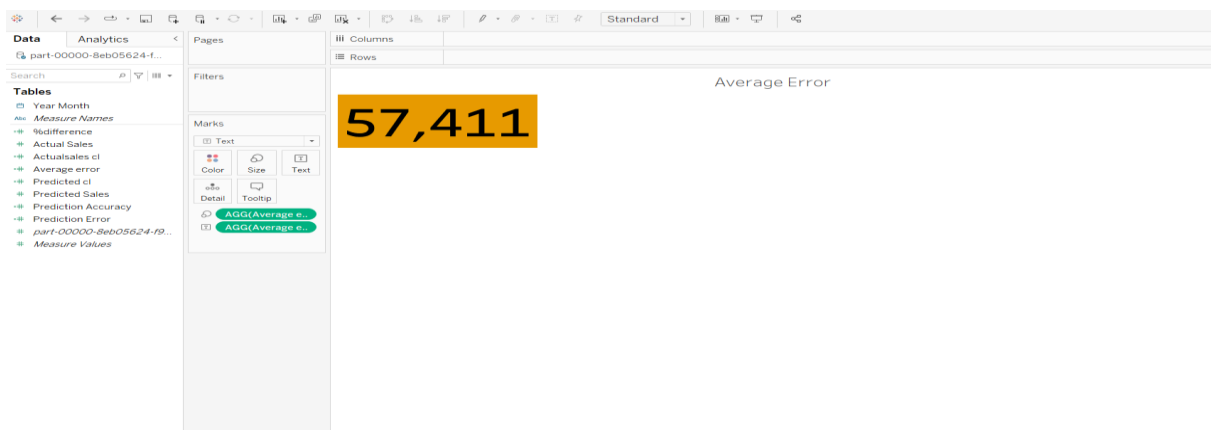
$ABS([Actual\_Sales] - [Predicted\_Sales])$



### 3.RSME



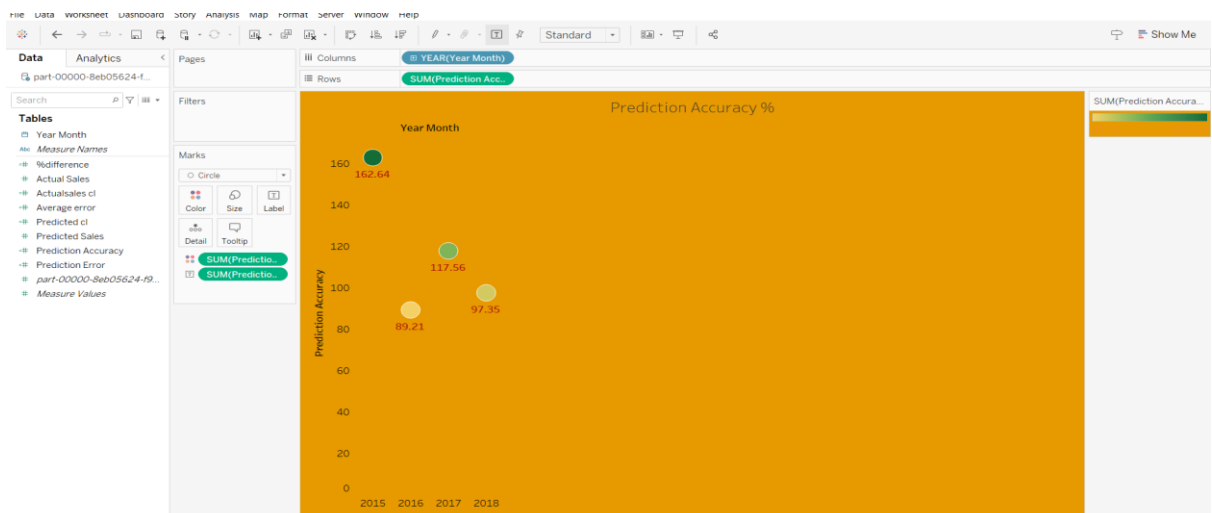
## 4.Average Error



## 5.Prediction Accuracy

Create Calculated Field named Prediction Accuracy

Write the formula  $(1 - \text{ABS}([\text{Actual Sales}] - [\text{Predicted Sales}]) / [\text{Actual Sales}]) * 100$

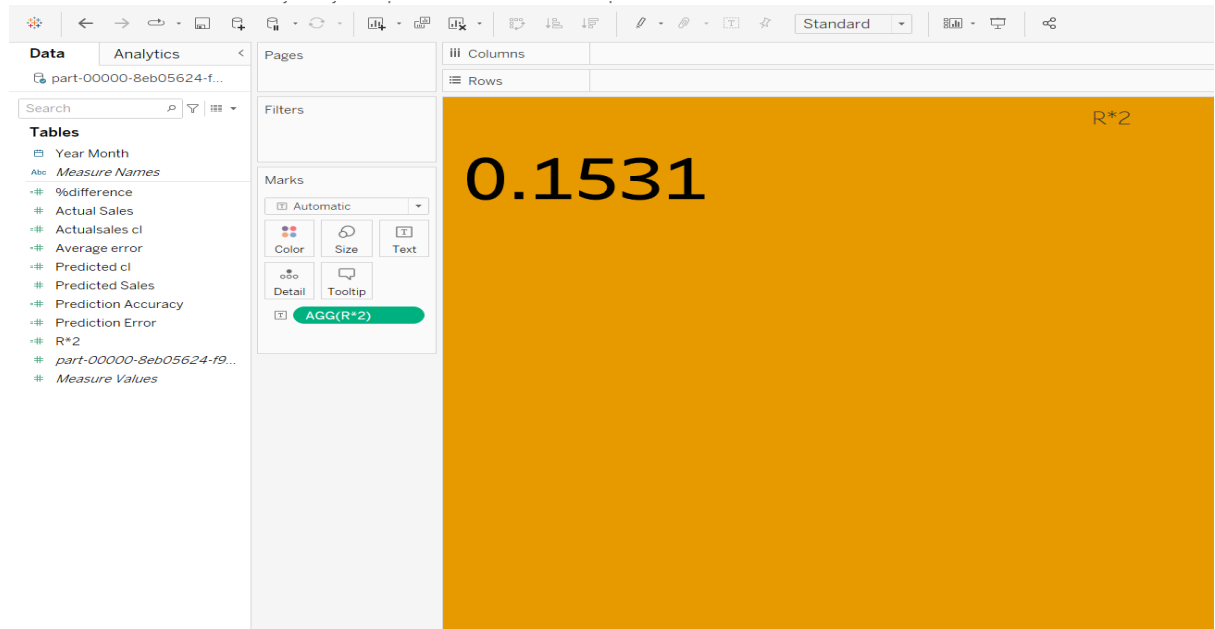


## 6.R\*2

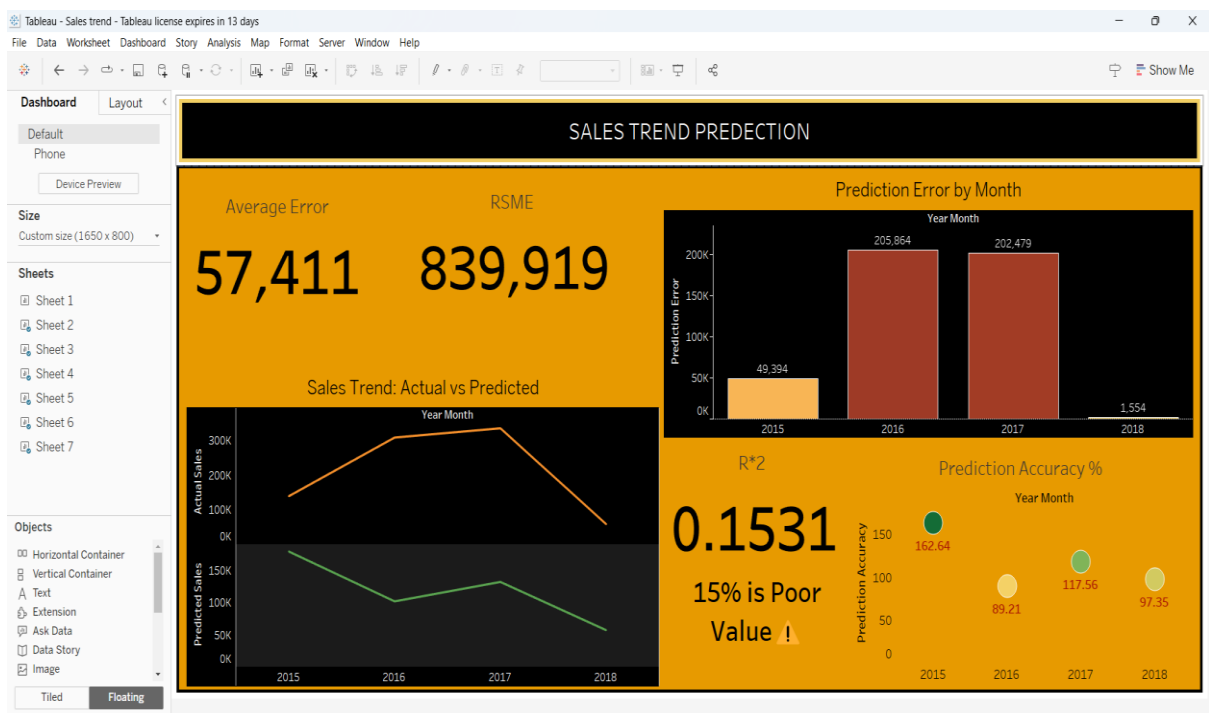
Create calculated field named as R\*2

And enter the formula

**SQUARE(CORR([Actual Sales], [Predicted Sales]))**



The Final Dashboard:-



## RESULTS:-

The Sales Trend Prediction project aimed to forecast future sales trends using historical sales data. After performing extensive data cleaning, preprocessing, and feature engineering, a Linear Regression model was trained using lag features such as previous month and 3-month sales values.

### Model Performance

The model achieved an **RMSE of approximately 839,919**, indicating that the average prediction error is about **₹839K**. While this demonstrates the model's capability to process large-scale sales data, the magnitude of error suggests significant room for improvement in prediction accuracy.

The  **$R^2$  score of 0.15** indicates that the model currently explains only **15% of the variance** in sales trends. This low  $R^2$  value reveals that **85% of sales variation remains unexplained** by the current features and modeling approach. While the model captures some basic historical patterns, it requires substantial enhancement to become a reliable forecasting tool.

### Current Limitations

- **Low predictive power:** The  $R^2$  of 0.15 is below industry standards (typically 0.70+ for acceptable models)
- **High prediction variance:** The RMSE suggests inconsistent accuracy across different time periods
- **Missing key drivers:** Critical factors influencing sales (seasonality, external events, market conditions) are not adequately captured
- **Limited feature set:** Reliance solely on lag features limits the model's explanatory capability